



目录

#	主题	内容
1~3	理论基础	FER 定义、心理学、FACS
4~6	模型体系	VGG → ResNet → MobileNet → MobileViT
7~9	数据与训练	FER-2013、训练流程、模型评估
10~12	部署与服务	ONNX、RESTful API、Flask
13~17	项目实操	环境搭建、训练、评估、API、前端

01 · 什么是 FER

人脸表情识别 (Facial Expression Recognition, FER)

“ 通过分析人脸图像或视频，自动识别面部表情所对应的情感状态 ”

技术交叉领域

计算机视觉 + 深度学习 + 情感计算 = FER

核心应用： 人机交互 · 教育监测 · 医疗辅助 · 驾驶安全 · 市场调研



FER 基本流程



FER 应用场景

场景	应用示例
人机交互 (HCI)	智能机器人理解情绪, 自适应交互
教育	在线课堂监测专注度与情绪状态
医疗健康	辅助抑郁症、自闭症早期筛查
驾驶安全	监测疲劳、愤怒等危险情绪
市场调研	分析消费者真实情感反应
安全监控	公共场所异常情绪预警

发展历程

1970s 萌芽期 Ekman FACS → 2000s 传统方法期 LBP/HOG/Gabor+SVM → 2013 深度学习期 CNN (VGG/ResNet) → 2020 现代前沿期 ViT/MobileViT

02 · 心理学理论基础

基本情绪理论 (Paul Ekman)

- 存在**跨文化、与生俱来**的基本情绪
- 每种基本情绪对应特定的面部表情模式
- 具有**跨文化一致性** (Fore 部落实验, 1971)

Ekman 六种基本情绪

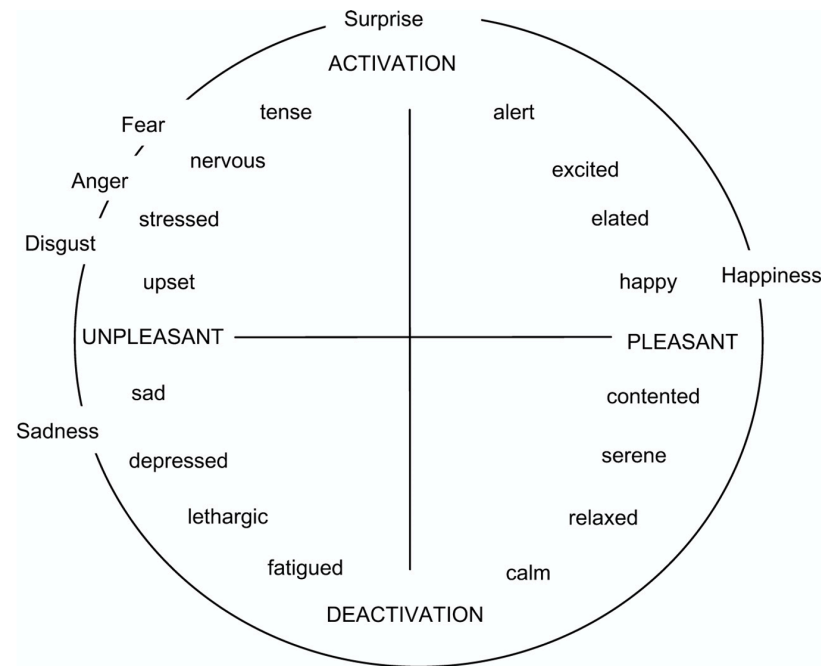
愤怒 (Anger)	厌恶 (Disgust)	恐惧 (Fear)	快乐 (Happiness)	悲伤 (Sad)	惊讶 (Surprise)
---------------	-----------------	--------------	-------------------	-------------	------------------

“ FER-2013 扩展为 **7 分类**：增加 **中性 (Neutral)** ”

维度情绪理论

Russell 环形情感模型 (Circumplex Model, 1980)

- 情绪在 **效价 (Valence)** × **唤醒 (Arousal)** 二维空间连续分布
- 离散基本情绪可映射到环形空间中



两种理论互补： 离散分类 → 标注依据； 维度模型 → 连续情感理解

03 · 七种基本表情

索引	表情	英文	核心特征	关键 AU
0	 愤怒	Angry	眉毛压低聚拢、 嘴唇紧闭	AU4+AU5+AU7+AU23
1	 厌恶	Disgust	鼻梁起皱褶 （最具标志性）	AU4+AU9+AU10
2	 恐惧	Fear	眼睛睁大 （眼白露出）	AU1+AU2+AU4+AU5+AU20+AU26
3	 快乐	Happy	眼角鱼尾纹 +嘴角上扬	AU6+AU12
4	 悲伤	Sad	嘴角下垂	AU1+AU4+AU15+AU17
5	 惊讶	Surprise	眉毛高高抬起成弓形	AU1+AU2+AU5+AU26
6	 中性	Neutral	面部肌肉放松， 无明显特征	无显著 AU

常见混淆模式

混淆对	原因分析	区分关键
恐惧 ↔ 惊讶	眉毛抬起 + 眼睛睁大 + 嘴巴张开	恐惧眉毛聚拢；惊讶眉毛分离
愤怒 ↔ 厌恶	都涉及眉毛压低	厌恶有鼻梁皱褶（AU9）
悲伤 ↔ 中性	悲伤强度弱时变化细微	悲伤眉头微翘（AU1）
恐惧 ↔ 悲伤	都有眉头内侧抬起	恐惧眼睛睁大，悲伤眼睛下垂

Duchenne 微笑 vs 社交微笑

“ 真正的快乐微笑伴随**眼轮匝肌（AU6）**收缩，产生眼角鱼尾纹；社交性微笑仅有嘴角上扬（AU12） ”

04 · 人脸动作编码系统 (FACS)

什么是 FACS

Facial Action Coding System — Paul Ekman & Wallace Friesen (1978)

“

任何面部表情都可分解为若干**动作单元 (AU) **的组合

”

上半脸主要 AU

AU	名称	关联情绪
AU1	内侧眉毛抬起	悲伤、恐惧
AU2	外侧眉毛抬起	惊讶、恐惧
AU4	眉毛压低	愤怒、厌恶
AU5	上眼睑抬起	恐惧、惊讶
AU6	脸颊抬起	快乐
AU9	鼻子皱起	厌恶标志

04 · 人脸动作编码系统 (FACS)

什么是 FACS

下半脸主要 AU

AU	名称	关联情绪
AU12	嘴角拉伸	快乐标志
AU15	嘴角下压	悲伤标志
AU20	嘴唇拉伸	恐惧
AU26	下颌下降	惊讶、恐惧

FACS → 七种表情映射

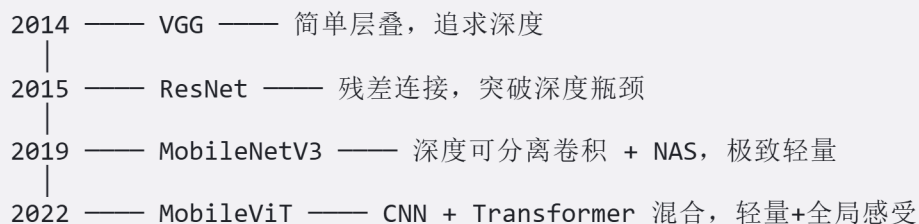
愤怒 (Angry)	= AU4 + AU5 + AU7 + AU23
厌恶 (Disgust)	= AU4 + AU9 + AU10
恐惧 (Fear)	= AU1 + AU2 + AU4 + AU5 + AU20 + AU26
快乐 (Happy)	= AU6 + AU12
悲伤 (Sad)	= AU1 + AU4 + AU15 + AU17
惊讶 (Surprise)	= AU1 + AU2 + AU5 + AU26
中性 (Neutral)	= 无显著 AU (基线状态)

FACS 在 FER 中的意义

- 🔍 **可解释性**：验证模型预测是否与 AU 特征一致
- 📋 **辅助标注**：为标注人员提供客观依据
- 🎯 **AU 检测任务**：从图像检测 AU → 映射情绪（更客观的路径）

05 · 深度学习 CNN 模型演进

四代模型脉络



特性	VGG-16	ResNet-50	MobileNetV3-S	MobileViT-XS
参数量	~138M	~25M	~2.5M	~2.3M
机构	牛津大学	微软研究院	Google	Apple
年代	2014	2015	2019	2022

VGG-16

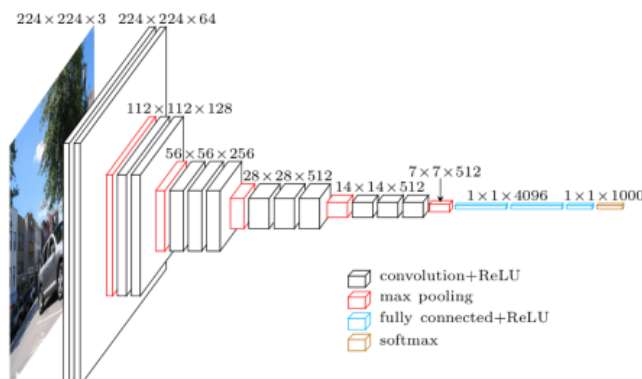
发布信息

项目	内容
论文	<i>Very Deep CNNs for Large-Scale Image Recognition</i>
发布者	Karen Simonyan, Andrew Zisserman
机构	牛津大学 Visual Geometry Group
年代	2014 (ICLR 2015)

核心设计

“ 用 3×3 小卷积核堆叠替代大卷积核，感受野相同但更深、更非线性 ”

7×7 Conv 感受野 = 3个 3×3 Conv 感受野
VGG 优势：更多 ReLU 激活 + 参数量更少 ($27C^2$ vs $49C^2$)



ResNet-50

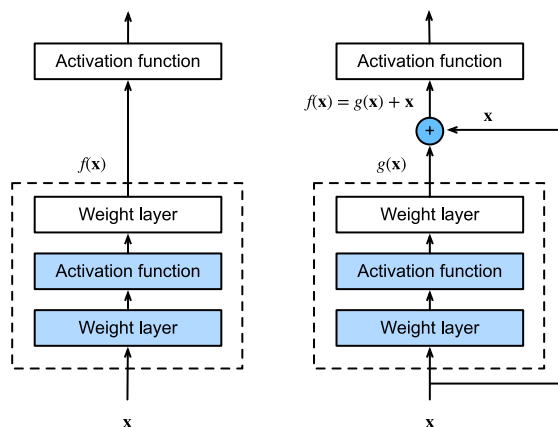
发布信息

项目	内容
论文	<i>Deep Residual Learning for Image Recognition</i>
发布者	Kaiming He (何恺明) 等
机构	微软研究院
年代	2015 (CVPR 2016 Best Paper)

核心创新：残差连接

传统 : $H(x)$ vs ResNet : $H(x) = F(x) + x$

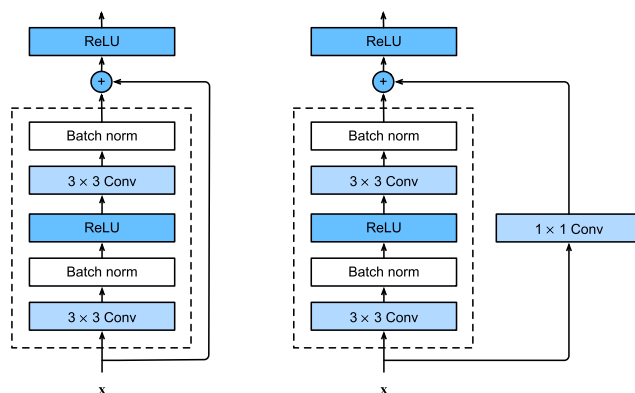
只需学习残差 $F(x) = H(x) - x$, 梯度通过跳跃连接直传



ResNet-50 · Bottleneck 设计

输入: 256-d

- 1×1 Conv, 64 ← 降维: 256→64
- 3×3 Conv, 64 ← 核心卷积
- 1×1 Conv, 256 ← 升维: 64→256
- + shortcut → ReLU



ResNet-50 结构速览

Conv1 (7×7,64) → Conv2_x (3 Bottleneck) → Conv3_x (4) → Conv4_x (6) → Conv5_x (3) → AvgPool → FC-7

变体	参数量	类型
ResNet-18	~11M	Basic Block
ResNet-50	~25M	Bottleneck
ResNet-152	~60M	Bottleneck

MobileNet

发布信息

版本	年代	核心创新	机构
MobileNetV1	2017	深度可分离卷积	Google
MobileNetV2	2018	倒残差 + Linear Bottleneck	Google
MobileNetV3	2019	NAS + SE + h-swish	Google

深度可分离卷积

标准卷积: $D_K^2 \times M \times N \times D_F^2$
↓
Depthwise: $D_K^2 \times M \times D_F^2$ (逐通道)
+
Pointwise: $M \times N \times D_F^2$ (逐点)
↓
计算量 $\approx$ 原来的 $1/9$ (对 3×3 卷积)

“ SE 模块: Squeeze (全局池化) → Excitation (学习通道权重)
→ 逐通道加权 ”

MobileViT

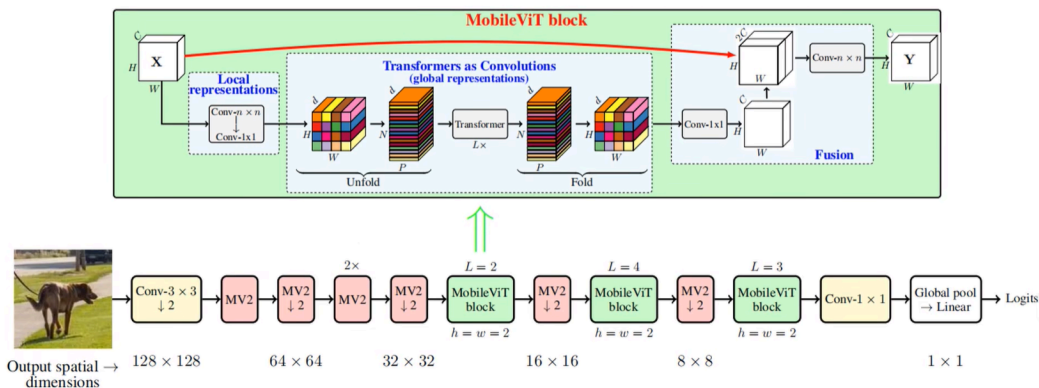
发布信息

项目	内容
论文	<i>MobileViT: Light-weight, General-purpose, Mobile-friendly ViT</i>
发布者	Sachin Mehta, Mohammad Rastegari
机构	Apple
年代	2022 (ICLR 2022)

CNN + Transformer 混合设计

输入 → [CNN 局部编码] → [MobileViT Block (Transformer 全局编码)] → [CNN 融合] → 输出

参数量仅 ~2.3M, ImageNet Top-1 达 74.8%



MobileViT Block 详解

核心机制

输入 [B,C,H,W]	
→ 3×3 Conv + 1×1 Conv	← CNN 局部编码
→ Unfold → [P, N, d]	← 展开为 Patch 序列
→ Transformer Encoder	← 跨 Patch 全局注意力
→ Fold → [B, d, H, W]	← 折回空间维度
→ 1×1 Conv + 3×3 Conv	← CNN 融合
→ + 残差连接 → 输出	

关键创新

- Transformer 在 **Patch 级别**做注意力，大幅降低计算量
- CNN 负责**局部细节**，Transformer 负责**全局关系**
- 极致的**参数效率**（每百万参数贡献约 30% 准确率）

四种模型对比总结

特性	VGG-16	ResNet-50	MobileNetV3-S	MobileViT-XS
核心思想	3×3 堆叠	残差连接	深度可分离 + SE	CNN+Transformer
参数量	138M	25M	2.5M	2.3M
模型大小	~512 MB	~90 MB	~5 MB	~7.5 MB
机构	牛津大学	微软研究院	Google	Apple
年代	2014	2015	2019	2022
推理(fps)	~67	~167	~333	~250
FER Best	70.79%	71.26%	69.24%	69.96%
参数效率	0.52	2.85	27.70	30.42

结论



MobileViT-XS 以 VGG **1/60** 的参数量达到其 **98.8%** 的准确率，是部署的**最优选择**



06 · FER-2013 数据集

属性	值
全名	Facial Expression Recognition 2013
来源	ICML 2013 Workshop
图像尺寸	48×48 像素, 灰度图
总样本数	35,887 张
类别数	7 类
标注方式	人工标注 (一致性约 65%)

类别分布

表情	训练集	占比
Happy	7,215	25.1%
Neutral	4,965	17.3%
Sad	4,830	16.8%
Fear	4,097	14.3%
Angry	3,995	13.9%
Surprise	3,171	11.0%
Disgust	436	1.5% ⚠

FER-2013 局限性分析

局限性	影响	缓解策略
类别不平衡	Disgust 几乎学不到	Focal Loss、重采样、数据增强
低分辨率 (48×48)	细节缺失	ImageNet 预训练权重
标注噪声	训练目标不一致	Label Smoothing
非人脸图像	学到错误特征	训练前过滤
水印/文字	依赖非表情特征	MixUp 增加多样性

数据目录结构

```
fer2013/
├── train/
│   ├── angry/      (3995 张)
│   ├── disgust/    (436 张)
│   ├── ...         (7 个类别目录)
├── test/
│   └── ...          (7178 张, 同类结构)
```

07 · Python 模型训练基础

核心依赖

库	用途
PyTorch	深度学习框架 (<code>nn</code> , <code>optim</code> , <code>DataLoader</code>)
torchvision	视觉模型与变换
NumPy / Pillow	数值计算 / 图像处理
scikit-learn	评估指标 (F1, Confusion Matrix)
tqdm / Matplotlib	进度条 / 可视化

关键概念

概念	解释
<code>epoch</code>	完整遍历训练集一次
<code>batch_size</code>	每次更新参数的样本数
<code>forward()</code>	数据流经网络得到输出
<code>backward()</code>	反向传播计算梯度
<code>optimizer.step()</code>	根据梯度更新参数

训练最小化示例

```
import torch, torch.nn as nn, torch.optim as optim
from torchvision.models import resnet50, ResNet50_Weights

# 1. 设备
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')

# 2. 模型（预训练 + 替换分类头）
model = resnet50(weights=ResNet50_Weights.IMAGENET1K_V1)
model.fc = nn.Sequential(nn.Dropout(0.2), nn.Linear(2048, 7))
model = model.to(device)

# 3. 损失 + 优化器
criterion = nn.CrossEntropyLoss()
optimizer = optim.AdamW(model.parameters(), lr=1e-4, weight_decay=1e-4)

# 4. 训练循环
for epoch in range(30):
    for images, labels in train_loader:
        images, labels = images.to(device), labels.to(device)
        optimizer.zero_grad()
        loss = criterion(model(images), labels)
        loss.backward()
        optimizer.step()
```


08 · 模型训练流程

三阶段全景

第1阶段：准备
数据准备 → 模型选择 → 超参配置

第2阶段：训练循环

```
for epoch in range(N):  
    | train() → forward → loss → bw  
    | eval() → 不计算梯度  
    | scheduler.step()  
    | EarlyStopping 检查  
    | 保存 best_model.pth
```

第3阶段：评估
测试集评估 → 混淆矩阵 → 错误分析

损失函数与正则化

损失函数对比

损失函数	公式	适用场景
CrossEntropy	$-\sum y_i \log(\hat{y}_i)$	类别均衡的标准分类
Focal Loss	$-\alpha_t(1 - p_t)^\gamma \log(p_t)$	类别不平衡 ($\gamma = 2.0$)
Label Smoothing	$y_i^{LS} = y_i(1 - \epsilon) + \epsilon/C$	防止过度自信

8 种正则化技术

技术	作用
Weight Decay / Dropout / Label Smoothing / MixUp	抑制过拟合
Focal Loss	抑制类别不平衡
Gradient Clipping	抑制梯度爆炸
Data Augmentation	增加数据多样性
Early Stopping	在验证集不改善时停止

09 · 模型评估

关键指标

准确率 Accuracy

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

⚠ 类别不平衡时不可靠

精确率 & 召回率

$$\text{Precision} = \frac{TP}{TP + FP} \quad \text{Recall} = \frac{TP}{TP + FN}$$

F1 分数

$$F1 = 2 \times \frac{P \times R}{P + R}$$

聚合方式

FER-2013 适用性

Macro F1

✓ **首选** (小类别权重与大类别相同)

Weighted F1

✗ Disgust (1.5%) 几乎无影响

混淆矩阵

FER-2013 典型混淆模式

真实↓	预测→						
	angry	disgust	fear	happy	sad	surprise	neutral
angry	[612	34	58	12	89	42	111]
disgust	[22	45	18	5	12	6	3]
fear	[45	12	523	23	89	278	54]
happy	[8	3	12	1523	56	45	127]
sad	[78	8	89	45	789	56	182]
surprise	[12	2	78	34	23	623	59]
neutral	[34	5	34	56	123	34	947]

混淆对

FACS 分析

fear ↔ surprise 均涉及 AU1+AU2+AU5

angry ↔ disgust 均涉及 AU4

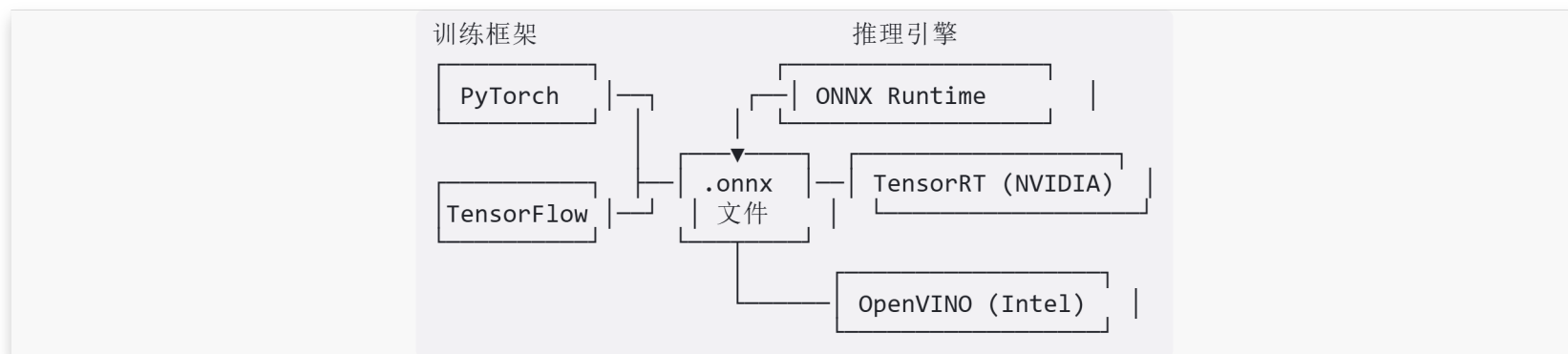
sad ↔ neutral 悲伤弱时 AU 不显著

happy 最易识别 AU6+AU12 组合独特

10 · ONNX 模型格式

什么是 ONNX

Open Neural Network Exchange — 开源模型交换格式标准



优势

优势	说明
框架无关	跨框架互操作
推理加速	比 PyTorch 快 1.5~3 倍
文件更小	移除梯度信息
跨平台	Windows/Linux/macOS/Android/iOS

PyTorch → ONNX 导出

```
import torch.onnx

model = create_fer_model('resnet', 'resnet50', num_classes=7)
model.load_state_dict(torch.load('best_model.pth'))
model.eval()

dummy_input = torch.randn(1, 3, 224, 224)

torch.onnx.export(
    model, dummy_input, "fer_model.onnx",
    export_params=True,
    opset_version=12,
    input_names=['input'],
    output_names=['output'],
    dynamic_axes={                                # 动态 batch
        'input': {0: 'batch_size'},
        'output': {0: 'batch_size'}
    }
)
```

各模型 ONNX 性能

模型	ONNX 大小	推理时间	FPS
VGG-16	~500 MB	~15 ms	~67
ResNet-50	~90 MB	~6 ms	~167
MobileNetV3	~5 MB	~3 ms	~333
MobileViT-XS	~7 MB	~4 ms	~250

11 · RESTful API 规范

REST 核心原则

原则	描述
资源	一切皆资源, URI 标识
无状态	每个请求自包含
统一接口	标准 HTTP 方法操作资源
分层系统	支持负载均衡、缓存

HTTP 方法与 CRUD

方法	CRUD	FER 项目示例
GET	Read	/api/models
POST	Create	/api/recognize
PUT	Update	/api/config
DELETE	Delete	/api/models/resnet

FER 项目 API 设计

FER API 服务
http://localhost:5000

GET	/api/health	→ 健康检查
GET	/api/models	→ 可用模型列表
POST	/api/model/switch	→ 切换模型
POST	/api/recognize	→ 表情识别（核心）
POST	/api/detect	→ 仅人脸检测
GET	/api/detector	→ 当前检测器
POST	/api/detector/switch	→ 切换检测器
GET	/api/benchmark	→ 性能基准

统一响应格式

```
{ "success": true, "data": { ... }, "message": "ok", "code": 200 }
```


12 · Flask Web 框架

为什么选 Flask

特性	Flask	Django
类型	微框架	全栈框架
学习曲线	平缓 ✓	较陡
适合场景	API 服务、原型 ✓	大型 Web 应用
灵活性	极高 ✓	遵循约定

最小应用

```
from flask import Flask
app = Flask(__name__)

@app.route('/')
def hello():
    return 'Hello, FER!'

app.run(host='0.0.0.0', port=5000)
```

“ FER API 是轻量级推理服务，Flask 的简洁性完美契合 ”

13 · 项目实操：环境搭建

环境准备

```
# 1. 创建虚拟环境
conda create -y -n face-fer python=3.10
conda activate face-fer

# 2. 安装 PyTorch (CUDA 11.8)
pip install torch==2.0.1+cu118 torchvision==0.15.2+cu118 \
  --index-url https://download.pytorch.org/whl/cu118

# 3. 安装项目依赖
pip install -r requirements.txt

# 4. 验证环境
python test/check_deps.py
```

依赖清单

类别	包名	用途
核心	numpy, pillow	数值计算、图像处理
模型	timm	MobileViT 模型库
推理	onnx, onnxruntime-gpu	模型导出与加速
Web	flask, flask-cors	API 服务
人脸	insightface	高精度人脸检测

项目结构一览

```
face-fer/
├── train.py / train_all.py  ← 训练入口
├── inference.py             ← 推理入口
├── run_api.py               ← API 启动入口
├── configs/                 ← 训练配置 (.json)
├── src/
│   ├── data/                ← 数据集加载 (fer2013, rafdb, ...)
│   ├── models/              ← VGG/ResNet/MobileNet/MobileViT
│   ├── train/               ← 训练器 + FocalLoss
│   ├── inference/           ← ONNX推理 + 人脸检测
│   └── api/                  ← Flask 路由 + 模型管理
├── frontend/                ← Web 前端 (HTML/CSS/JS)
├── checkpoints/             ← 训练输出 (best_model.pth)
└── dataset_data/fer2013/    ← 数据集
```

关键设计模式

模式	位置	作用
工厂模式	factory.py	统一创建任意模型
策略模式	losses.py	可插拔损失函数
配置驱动	configs/*.json	不改代码改配置

14 · 项目实操：训练

单模型训练

```
# MobileNetV3 (最快, ~15分钟)
python train.py --config configs/train_mobilenet_fer2013.json

# ResNet-50 (最佳精度)
python train.py --config configs/train_resnet_fer2013.json

# VGG-16 / MobileViT-XS
python train.py --config configs/train_vgg_fer2013.json
python train.py --config configs/train_mobilevit_fer2013.json
```

批量训练

```
python train_all.py
```

训练日志解读

```
Epoch 12/50
  Train Loss: 0.8523, Train Acc: 76.45%
  Val Loss: 1.1234, Val Acc: 65.32%
  LR: 0.000245, Time: 45.2s
```

训练配置详解

```
{
  "training": {
    "epochs": 50,
    "batch_size": 32,
    "optimizer": "AdamW",
    "lr": 3e-4,
    "weight_decay": 1e-4,
    "scheduler": "CosineAnnealingWarmRestarts",
    "warmup_epochs": 5,
    "T_0": 25, "T_mult": 2,
    "label_smoothing": 0.1,
    "use_focal_loss": true,
    "mixup_alpha": 0.2,
    "early_stop_patience": 15,
    "use_amp": true,
    "grad_clip": 1.0
  }
}
```

历史训练结果

模型	Best Val Acc	参数效率
ResNet-50	71.26%	2.85
VGG-16	70.79%	0.52
MobileViT-XS	69.96%	30.42
MobileNetV3	69.24%	27.70

15 · 项目实操：推理与 ONNX

单张推理

```
# PyTorch 推理
python inference.py --model checkpoints/xxx/best_model.pth --image test.jpg

# ONNX 导出
python inference.py --export checkpoints/xxx/best_model.pth \
  --model models/fer.onnx

# ONNX 推理
python inference.py --model models/fer.onnx --image test.jpg

# 性能基准 (1000次)
python inference.py --model models/fer.onnx --benchmark 1000
```

推理输出示例

```
识别结果:
表情: happy
置信度: 0.9523
各类别概率:
  angry: 0.0012  disgust: 0.0003  fear: 0.0021
  happy: 0.9523  sad: 0.0015    surprise: 0.0387
  neutral: 0.0039
```

16 · 项目实操：API 服务化

启动服务

```
python run_api.py
```

```
=====
表情识别服务
管理界面: http://0.0.0.0:5000
API文档:  http://0.0.0.0:5000/api/docs
健康检查: http://0.0.0.0:5000/api/health
=====
```

测试 API

```
# 健康检查
curl http://localhost:5000/api/health

# 表情识别
curl -X POST http://localhost:5000/api/recognize \
-F "image=@test_face.jpg"

# 切换模型
curl -X POST http://localhost:5000/api/model/switch \
-H "Content-Type: application/json" \
-d '{"model_name": "mobilenet"}
```

API 架构图

HTTP Request —▶

Flask Server

src/api/routes.py

- └ /api/health
- └ /api/recognize
- └ ...

src/api/model_manager.py

- └ 扫描 checkpoints/
- └ 预转换全部 ONNX
- └ 模型加载/切换/推理

src/inference/detector.py

- └ InsightFace (高精度)
- └ OpenCV (轻量兼容)

src/inference/inferer.py

- └ ONNX Runtime 推理

17 · 项目实操：前端平台

访问地址

启动 API 后访问 <http://localhost:5000>

前端功能

区域	功能
模型选择	下拉切换 VGG/ResNet/MobileNet/MobileViT
检测器切换	InsightFace (高精度) / OpenCV (轻量)
图片上传	拖拽上传 / 点击上传 / Ctrl+V 粘贴
人脸检测	Canvas 标注人脸框
表情识别	表情类别 + 置信度 + 概率分布柱状图

技术栈

index.html (页面) + style.css (样式) + app.js (交互)
⬆ Fetch API ⬆
Flask RESTful API

课程总结

你已掌握的技能

环节	技能点
 理论	FER 定义、Ekman 情绪理论、FACS 编码
 模型	VGG → ResNet → MobileNet → MobileViT
 数据	FER-2013 分析、类别不平衡处理
 训练	PyTorch、Focal Loss、MixUp、早停
 评估	Accuracy/F1/混淆矩阵/过拟合诊断
 部署	ONNX 导出、Flask API、Web 前端

关键技术要点回顾

三个"最优"

1. **精度最优** — ResNet-50 (Val Acc **71.26%**)
2. **参数效率最优** — MobileViT-XS (**30.42** Acc/M 参数)
3. **部署最优** — MobileViT-XS (2.3M 参数, ~250 FPS)

核心技术路线

Ekman 基本情绪理论 → FACS 动作编码 → CNN/Transformer 模型
→ FER-2013 训练 → ONNX 部署 → Flask API → Web 前端

参考资料

- Ekman, P. & Friesen, W. V. (1978). *Facial Action Coding System*
- Simonyan & Zisserman (2014). *VGG*. ICLR 2015
- He, K. et al. (2016). *ResNet*. CVPR 2016 Best Paper
- Howard, A. et al. (2019). *MobileNetV3*. ICCV 2019
- Mehta & Rastegari (2022). *MobileViT*. ICLR 2022
- Goodfellow, I. J. et al. (2013). *FER-2013*. ICML Workshop
- **ONNX**: <https://onnx.ai/>
- **Flask**: <https://flask.palletsprojects.com/>

Thank You

准备好了吗？开始动手实践！

```
python train.py --config configs/train_mobilenet_fer2013.json
```

“

📖 配套文档：

- 基础知识手册
- 实操手册
- 技术架构说明

”